CHAPTER 18

Agile Architecture



This chapter is part of the BLUE reading tour. See Figure 1-1 in Chapter 1, Section 1.3, for alternative reading tours.

Agile architecture is not an oxymoron. It is a conscious approach to designing and developing architecture in a way that embraces change, fosters collaboration, and delivers value incrementally. In the context of the TOGAF Standard, agile architecture represents a harmonious blend of structured methodology and adaptive practices.

The TOGAF Standard provides a robust framework that can be tailored to support agile principles. By incorporating iterative cycles, continuous stakeholder engagement, and incremental delivery, the Architecture Development Method can be adapted to facilitate agility within Enterprise Architecture initiatives. This adaptation ensures that architecture development is responsive to evolving business needs while maintaining alignment with strategic goals.

This chapter examines how the TOGAF Standard accommodates agile architecture. It explores the integration of agile practices within the Architecture Development Method phases and discusses the concept of just enough architecture. Through this exploration, it is demonstrated that the TOGAF Standard is not only compatible with agile methodologies, but also enhances their effectiveness by providing a structured approach to managing complexity and change.

© Eric Jager 2025
E. Jager, *Mastering the TOGAF® Standard*, https://doi.org/10.1007/979-8-8688-1814-1_18

18.1. What Is Agile Architecture

The term *agility* is often thrown around in different contexts. But what does it really mean? While interpretations may vary, certain characteristics consistently emerge when discussing agility.

At its core, agility is about *adaptability*. It's a mindset that *anticipates change* and adapts accordingly. It involves short, iterative cycles and frequent reassessment of priorities to ensure that organizations remain responsive and relevant in dynamic environments.

Agility emphasizes the *delivery of tangible value*. Activities are driven by their potential to have the greatest impact. This approach involves continually evaluating and reprioritizing tasks to focus on high-value outcomes, often minimizing intermediate products and extensive documentation. Agility encourages a culture of experimentation. Rather than exhaustive theoretical analysis, it encourages trying ideas, learning from results, and iterating quickly. This *fail-fast* approach fosters innovation and continuous learning.

Agile environments thrive on the autonomy of skilled, multidisciplinary teams. These teams are empowered to make decisions, which promotes a sense of ownership and accountability for their results.

Agility is a journey, not a destination. It embodies an ongoing commitment to refining processes, improving performance, and striving for excellence. This relentless pursuit of improvement ensures that organizations evolve and adapt over time.

Agile means to move/change quickly and easily, often to provide value-generating outcomes [23].

Today, when the word *agile* is mentioned, many people immediately think of the *Agile Software Development Process*, which is rooted in the "Manifesto for Agile Software Development" [26]. While these principles are invaluable, it's important to recognize that organizational agility encompasses a broader spectrum than just agile methodologies.

18.1.1. The Basic Concepts Explained

The approach to developing and ultimately implementing an agile architecture requires an understanding of a few concepts. So let's start with the basics. Figure 18-1 shows a conceptual visualization of an architecture. This architecture is divided into three levels of granularity (Strategic, Segment, and Capability Architectures). One of the levels (the Capability Architecture) is broken down into three smaller pieces.

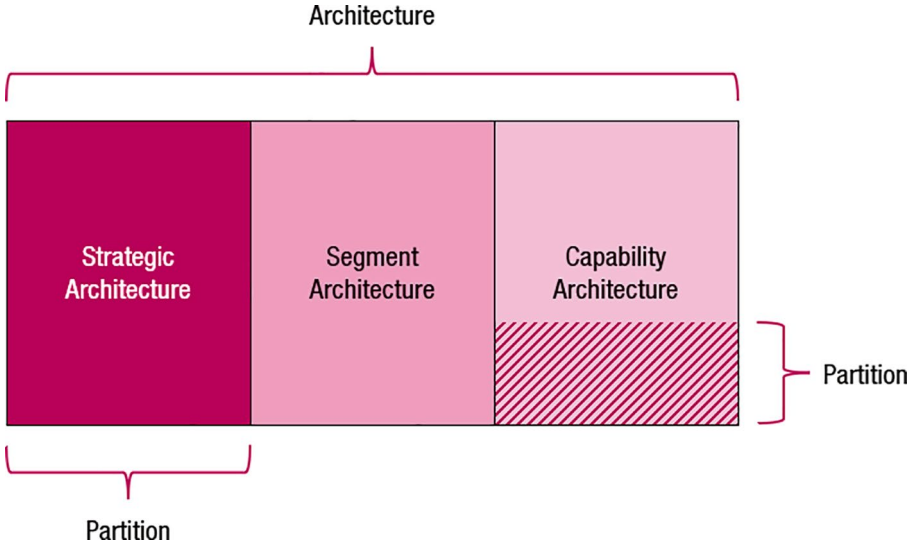


Figure 18-1. *Architecture divided into partitions*

Splitting an architecture or level of architecture granularity into one or more smaller parts is called *architecture partitioning*. This concept is explained in Chapter 13, Section 13.4. An architecture partition is a logical scope of work that can be independently architected and managed. In layman's terms, an architecture partition refers to *a smaller piece of the overall architecture*.

Architecture partitions can be developed and implemented using *agile sprints*. A sprint is a short, time-bound iteration that delivers working functionality. In agile environments, sprints are often used to deliver features or increments within a partitioned architecture. As shown in Figure 18-2, each architecture partition can be mapped to one or more agile sprints that focus on building solutions within that partition.

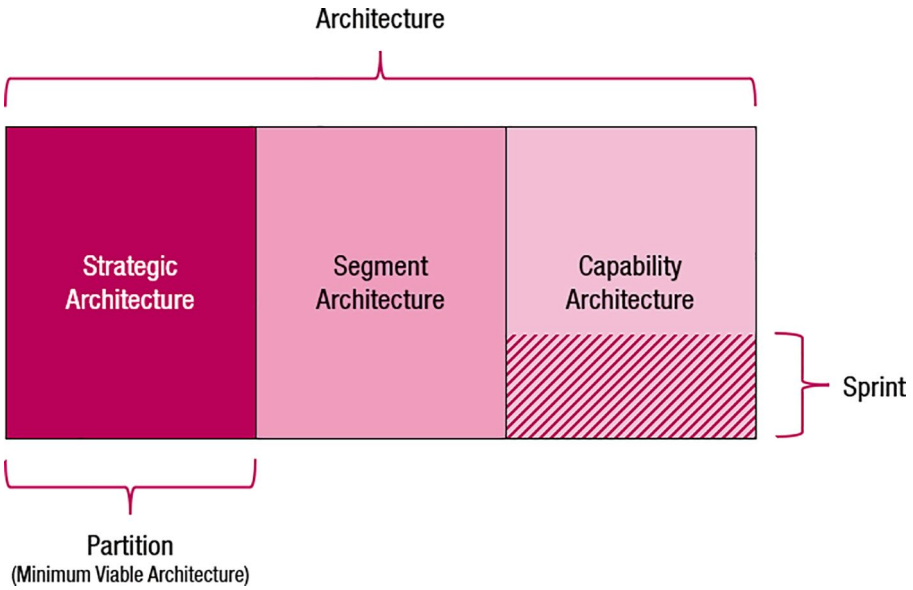


Figure 18-2. *Architecture showing partitions and sprints*

An architecture partition contains at least the minimum architecture that is realizable and adds business value. In agile terms, this is called a *Minimum Viable Architecture*.

A Minimum Viable Architecture defines the minimum architecture that is realizable and add business value [23].

In essence, an agile approach in the context of architecture means doing work in manageable chunks. Where architecture using a non-agile approach is often seen as a long process, an agile approach is about being able to deliver pieces of the whole quickly and in short cycles. Each piece is developed and delivered in one or more sprints (short, time-bound iterations that deliver working functionality). An important point to note here is that the work does not have to be done sequentially, as is common in a waterfall approach. An agile approach supports doing work in *parallel*. But with the understanding that the big picture must not be lost.



Figure 18-3. *Regular approach to architecture development*

The regular approach to architecture development, shown in Figure 18-3, consists of five components:

- Define Problem
- Define Baseline
- Define Target
- Deploy Target
- Govern and Manage Change

The five components listed above are directly related to the phases of the Architecture Development Method. For example, the first component, *Define Problem*, is addressed in Phase A (Architecture Vision). The *Define Baseline* and *Define Target* components are touched on during phases B–D (Business Architecture, Information Systems Architectures, and Technology Architecture). The *Deploy Target* section relates to Phase E

(Opportunities and Solutions), Phase F (Migration Planning), and Phase G (Implementation Governance). Finally, Phase H (Architecture Change Management) can be seen in the *Govern and Manage Change* section.

As mentioned above, an agile way of working does not use a *sequential* approach. Instead, its strength lies in the ability to perform short-cycle tasks (sprints) in *parallel*. During each of the components described above, one or more sprints take place. Each sprint accomplishes a *portion of the total work* to be done for a component. Sprints can run in parallel. This is visualized in Figure 18-4.

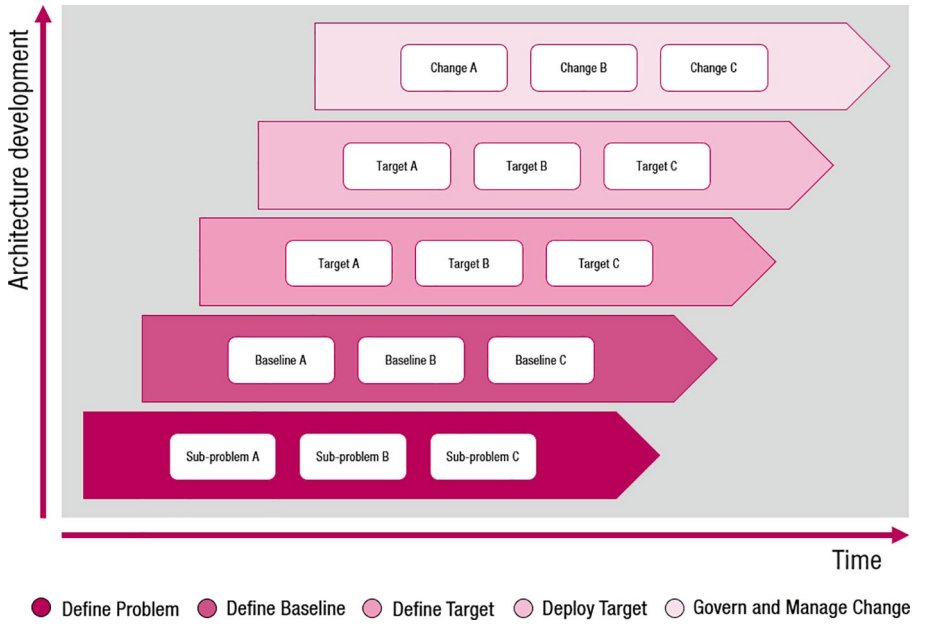


Figure 18-4. *Components of a regular approach that includes sprints*

It is by no means necessary to complete the problem definition before beginning other activities. It is sufficient to define enough of the problem to provide a context for other activities. As these other activities are initiated, work can continue to refine and extend the problem definition.

There are of course obvious dependencies. It is not possible to implement any part of the target until the parts of the baseline and target that describe that particular part of the target have been sufficiently defined.

It's like creating a painting. If you don't know what you want the painting to look like, you cannot start painting.

The TOGAF Standard embraces agile working through the use of architecture partitioning. Architecture partitioning is the process of dividing architecture work into smaller chunks across multiple teams. Along with partitioning the workload, definitions are provided so that each team understands what they need to create in terms of the architecture. The architecture is developed and implemented by performing sprints.

18.2. Three Levels of Architecture Granularity

Remember the breakdown of the architecture levels that were described in Chapter 7, Section 7.2, that existed in the Architecture Landscape? The TOGAF Standard recognizes a model that identifies three levels of detail that can be used to partition the architecture development (see also Figure 18-5):

- **Strategic Architectures:** Provide a long-term summary view of the entire organization. These architectures provide an organizing framework for operational and change activities and enable executive-level direction setting.
- **Segment Architectures:** Provide more detailed operating models for areas within an organization. Segment Architectures can be used at the program or

portfolio level to organize and operationally align more detailed change activities. Another term often used for Segment Architectures is Domain Architectures.

- **Capability Architectures:** Show in more detail how the organization can support a particular unit of capability. Capability Architectures are used to provide an overview of current capability, target capability, and capability increments and allow individual work packages and projects to be grouped into managed portfolios and programs.

Referring back to Figure 18-3 in Section 18.1, the *problem* illustrated in the figure is defined at the strategic and segment levels. At the segment and capability levels, a *target* is defined, and at the capability level a target is *deployed* by agile teams supported by the architect and the Capability Architecture definition.

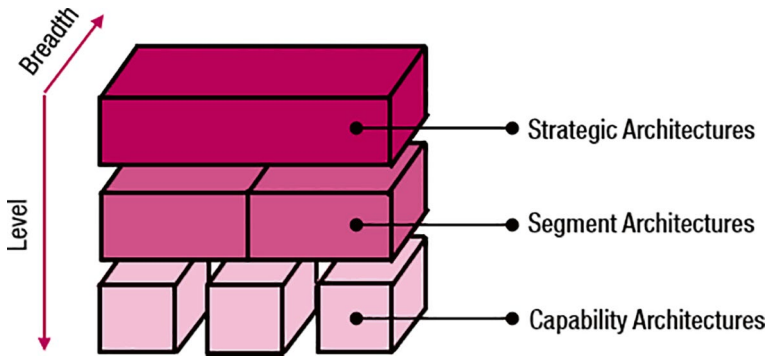


Figure 18-5. *Three levels of architecture granularity*

Strategic Architectures: Serve as the compass that guides an organization's long-term vision. They provide a broad, high-level perspective that contextualizes different segments and capabilities within the organization. This overarching view is essential for orchestrating comprehensive planning and design efforts, thereby mitigating the risk of unanticipated outcomes.

In agile organizations, Strategic Architecture is not a static blueprint, but rather a *dynamic iteration*. It is supported by Phase A (Architecture Vision) of the Architecture Development Method, which sets the strategic direction to inform decision-making processes. This strategic iteration can be further elaborated in Phase B to provide a more detailed view of the organizational landscape. It also helps to identify the necessary architecture and solution delivery capabilities required to achieve the organization's objectives.

Segment Architectures: Typically operate at the portfolio, program, or product level. These segments are not arbitrary; they are aligned with the structural contours of the organization, providing clarity and focus.

In the realm of Segment Architecture, the principle of *just enough* is paramount. It's not about exhaustive detail, but about identifying key features and both functional and non-functional requirements. This approach doesn't require going through every phase and step of the Architecture Development Method. Instead, it emphasizes the minimum necessary to ensure effective results.

Multiple iterations can be delivered simultaneously by different agile teams, fostering a dynamic and responsive development environment. Involving delivery teams in defining Segment Architectures creates a common understanding of the solution, setting the stage for cohesive and efficient implementation.

At its core, Segment Architecture is about strategic precision, providing just enough detail to guide development without stifling agility. It's a testament to the power of focused, collaborative design to achieve business goals.

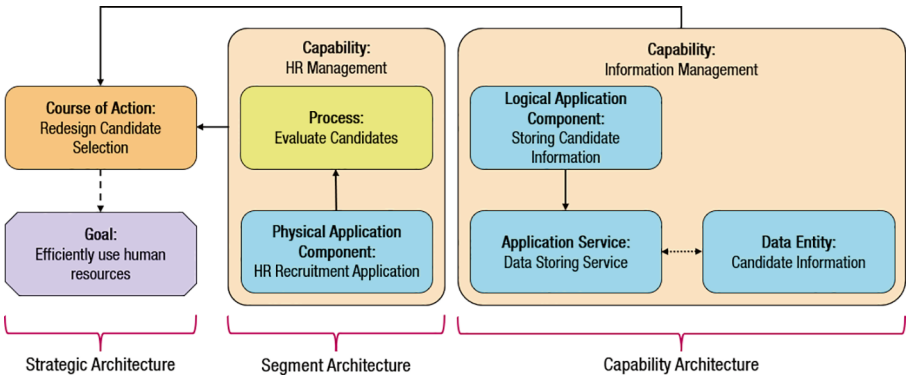


Figure 18-6. *Strategic, Segment, and Capability Architectures*

Capability Architectures: Serve as the essential link between high-level strategy and hands-on implementation. Think of them as the detailed blueprints that guide delivery teams as they bring business capabilities to life. They delve into the specifics of business capabilities and break them down into manageable increments. Each increment is meticulously detailed, ensuring that developers have clear, actionable guidance. Whether a capability is aligned with a single delivery sprint or spans multiple sprints, the focus remains on delivering tangible value.

It's worth noting that while sprints can occur at various levels, they are most commonly aligned with the delivery of capabilities or their increments. This alignment ensures that each sprint contributes meaningfully to the overarching business goals.

In addition, Capability Architectures don't exist in isolation. They build on Segment Architectures, adding layers of detail to ensure that solutions are not only well designed, but also implementable. This layered approach ensures that each architectural component is seamlessly aligned, facilitating a cohesive and effective implementation strategy.

Figure 18-6 shows that the Capability Architecture to be developed, which consists of three components, is part of the business capability Information Management. Partitioning the work associated with this

architecture results in the three components being developed separately. Each sprint focuses on developing a specific part of the total. An example of this is shown in Figure 18-7.

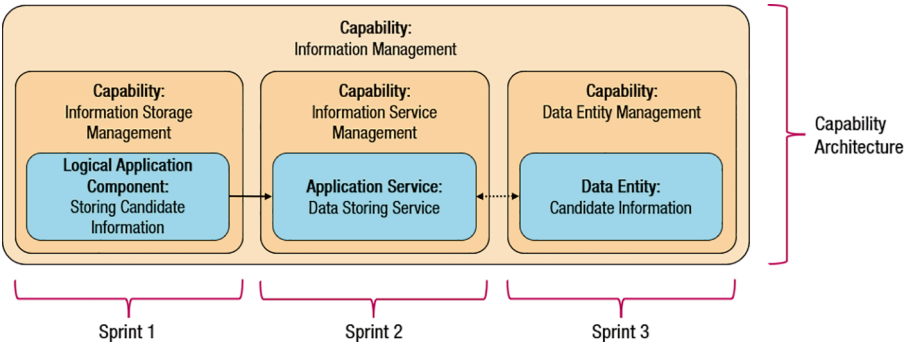


Figure 18-7. *Development of a partitioned architecture using three sprints*

The Capability Architecture from Figure 18-6 is shown in Figure 18-7 in a zoomed view. Clearly shown is the implementation of the development of a *logical application component*, an *application service*, and a *data entity*. Each component is implemented during a separate sprint.

Mapping architecture partitions to agile sprints is described in the next section.

18.3. Architecture Partitions and Agile Sprints

The idea behind an agile approach is not to use the waterfall principle and wait to do work until all previous issues have been completed. The development of Segment Architectures can therefore begin as soon as all relevant areas of interest are known. Even if not everything has been worked out in detail, the architecture work can begin. A similar approach applies to Capability Architectures. The development of these

architectures does not have to wait for the completion of the Segment Architectures. As illustrated in Figure 18-4 in Section 18.1.1, much of the architecture work and development can be done in parallel.

Of course, it is important not to lose sight of the big picture. The relationship to the organizational strategy must be maintained at all times. The advice is to strike a balance between providing enough detail at each level and providing too much detail too early without the added value of feedback from previous sprints. Developing an architecture that reflects exactly what is needed at the time – and nothing more – is often called a *Minimum Viable Architecture*.

It is important to identify the segments that need to be involved. This will largely determine the scope of the Segment Architecture. Once this scope is clear, it can be further refined into Capability Architectures. These can be implemented using an agile approach. The incrementally delivered Capability Architectures can be used to iteratively construct the final outcome (product, service, or solution). These iterations can be performed in parallel, depending on the degree of interdependency between them. Capability Architectures corresponding to a Segment Architecture can also be executed in parallel.

The end result of the Capability Architecture is the solution specification that is built and deployed on demand by agile teams according to the architecture guidelines, metrics, and compliance considerations.

The TOGAF Standard recognizes the need to recursively partition the Enterprise Architecture into more granular levels (Strategic, Segment, and Capability Architectures) that can be architected using an agile approach.

Partitioning the architecture work is key to agile delivery and involves defining segmentation based on the Strategic and Segment Architecture concepts discussed earlier. These smaller pieces, covering a specific area of the organization, can then be more easily specified and implemented using an agile approach. Partitions are considered to be the agile equivalent of a *Minimum Viable Architecture*.

Let's say that an average architecture project is being developed. The architecture work consists of the Baseline Architecture, a Transition Architecture, and a Target Architecture (see Figure 18-8).

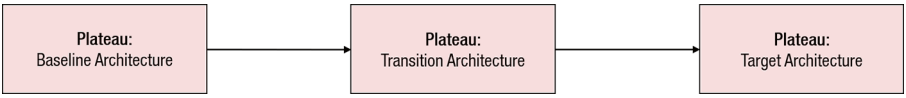


Figure 18-8. *Architecture project showing Baseline, Transition, and Target Architecture plateaus*

Using architecture partitioning (see Chapter 13, Section 13.4), the work packages or projects containing the activities necessary to create the Transition Architecture could be split (partitioned) into smaller pieces, creating multiple Transition Architectures, as demonstrated in Figure 18-9.

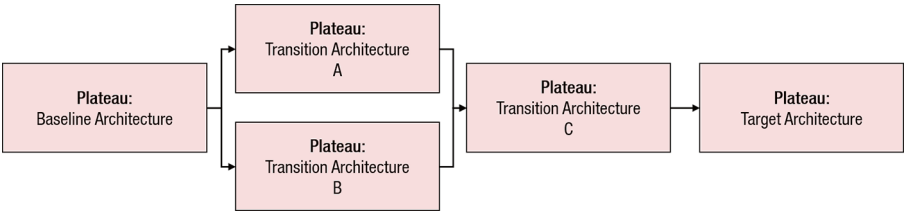


Figure 18-9. *Architecture project showing Baseline Architecture, partitioned Transition Architectures, and Target Architecture plateaus*

Figure 18-9 shows the application of partitioning to the architecture development design shown earlier. Here, the Transition Architecture (still referred to as a single plateau in Figure 18-8) is partitioned into several Transition Architectures (A, B, and C).

> A Transition Architecture is a formal description of one state of the architecture at an architecturally significant point in time, usually including a number of capability increments [23].

Each of the shown Transition Architectures (A, B, and C) has its own architecture to be developed and implemented. Figures 18-7 and 18-9 are combined in Figure 18-10, which shows the architecture to be implemented per sprint as part of each individual Transition Architecture.

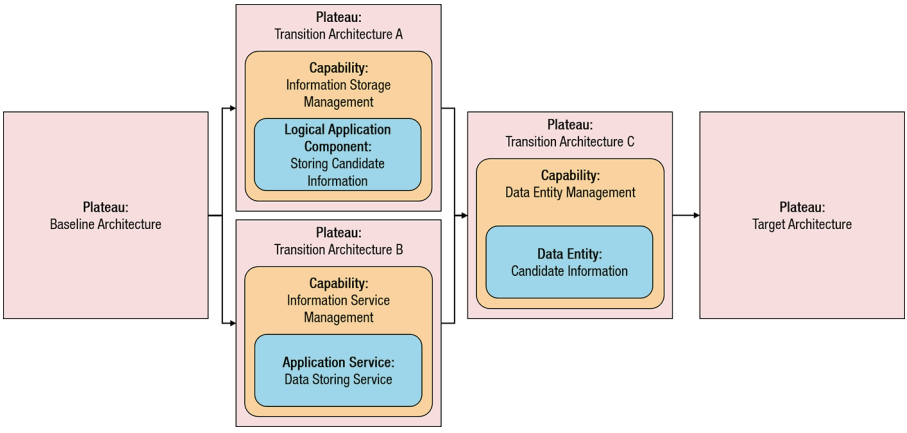


Figure 18-10. *Architecture project showing Transition Architectures and sprints*

Splitting or partitioning the work associated with the Transition Architecture into multiple Transition Architectures creates smaller pieces of deliverable architecture. These pieces (partitions) can be developed and implemented using one or more agile sprints.

Now, combining the individual sprints from Figure 18-10 and the basic concepts from Figure 18-2 (Section 18.1), it is possible to see how the sprints relate to these basic concepts. This is shown in Figure 18-11.

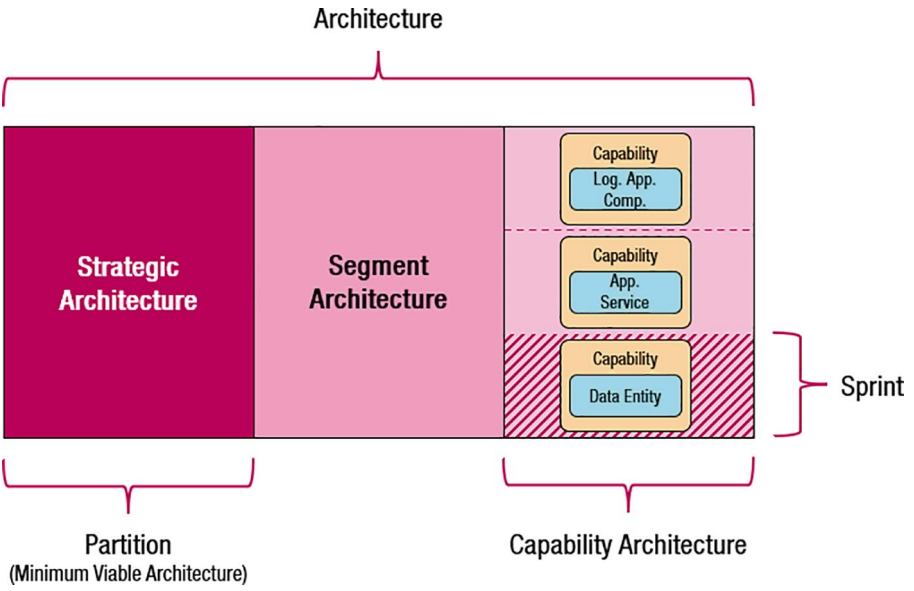


Figure 18-11. *Architecture showing partitions and detailed sprints*

The TOGAF Standard fully supports an agile way of working. The Architecture Development Method can be easily adapted to support agile concepts, and the phases of the Architecture Development Method can be mapped to them. It is also quite possible to indicate for each phase of the Architecture Development Method what needs to be done in relation to the execution of agile sprints.

18.4. Developing an Agile Architecture

The TOGAF Standard supports an agile approach through the use of the Architecture Development Method. The three levels of granularity and associated phases of the Architecture Development Method map well to the concepts of an agile approach. Due to its iterative nature, this approach can be used very effectively to interpret the relationship to the agile concepts.

The phases of the Architecture Development Method can also be aligned with an agile approach. Each phase separately supports the agile approach, making it possible to apply the architecture approach of the TOGAF Standard in an agile environment.

18.4.1. Mapping the ADM to Agile Concepts

Section 18.2 described the three levels of granularity recognized by the TOGAF Standard (Strategic, Segment, and Capability Architectures). These three levels can be used to develop the architecture in parallel using different iterations. This approach is equivalent to developing an architecture using techniques such as SAFe and Scrum.

Table 18-1 shows how the three levels of architecture granularity, combined with the corresponding phases of the Architecture Development Method, can be mapped to different agile concepts.

Table 18-1. *Architecture granularity levels and ADM Phases mapped to agile concepts*

Architecture Granularity Level	ADM Phase	SAFe/Agile/Scrum Layer
Strategic Architecture	Preliminary Phase Phase A: Architecture Vision	SAFe – Strategic Themes – Enterprise Portfolio Delivery
Segment Architecture	Phase B: Business Architecture Phase C: Information Systems Architectures Phase D: Technology Architecture	Agile/SAFe – Product/Portfolio Backlog – Change Plans – Epics

(*continued*)

Table 18-1. (*continued*)

Architecture Granularity Level	ADM Phase	SAFe/Agile/Scrum Layer
Capability Architecture	Phase E: Opportunities and Solutions Phase F: Migration Planning Phase G: Implementation Governance Phase H: Architecture Change Management	Agile/SAFe – Product/Solution Backlog – Capabilities to be delivered Agile/SAFe – Product/Program Backlog – Capability Increments (Features) Agile/SAFe – Program Team Backlog – Deliverable User Stories Scrum – Sprint Team Backlog – Deliverable User Stories

At the *strategic level*, the focus is more on the Preliminary Phase (if changes to the architecture capabilities are required) and Phase A to provide the basis for defining the cross-organizational and strategic change time horizon view. This results in a set of high-level strategic plans known as *courses of action*.

Agile techniques typically address this with concepts such as high-level strategic themes and the highest level of an organization's product portfolio backlog. At this level, cross-functional teams (business, technical, design, implementation, and operations) must be involved to develop an Enterprise Architecture that meets both the business goals and objectives of the organization and is also potentially deliverable.

At the *segment level*, the focus is on *partitioning the courses of action* among the relevant organizational units so that the work of implementing the change can be organized effectively and efficiently. If the information gained from performing Phase A is not sufficient for this activity, the focus may be on exploring phases B, C, and D in more detail.

The work can be approached by factoring the work into self-organizing teams at different levels (in accordance with the chosen organizational unit structure), along with a high-level iteration through phases C and D that provides more detailed information for product or solution delivery, going deeper into smaller organizational areas (segments). The outputs of this iteration are the Epics, which reflect large or long-running user stories, and the segment-based initial portfolio and/or backlog. The output from this level can be used to test and experiment with new products (if necessary), providing descriptions for prototypes to test ideas in the relevant segment market.

At the *capability level*, phases B, C, and D identify a more specific solution-oriented architecture specification, including the Architecture Building Blocks, covering both the functional and non-functional aspects of the solution to be implemented. These architecture specifications are then further developed in phases E and F as the basis for the Solution Building Blocks and their integration into the desired solutions, services, and/or products.

This approach to rapid, continuous implementation at the lowest level of capability creates a Transition Architecture of deliverable units, similar to the sprints at the lowest level of the backlog in agile-style approaches. This smallest level of capability is often referred to as the *Minimum Viable Product* in agile approaches.

A Minimum Viable Product is an output that satisfies a minimum set of functional and non-functional requirements and can be realized when implemented in a live operational environment [23].

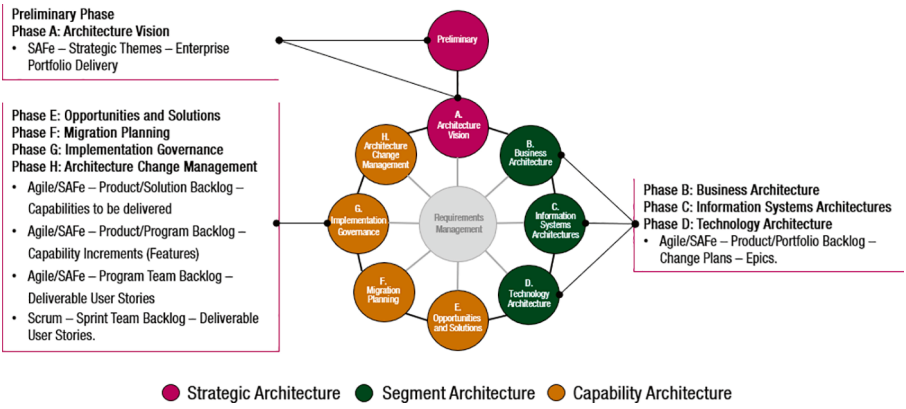


Figure 18-12. *The ADM mapped to agile concepts*

The backlogs shown in Figure 18-12 are not simple lists of activities. They are refined to reflect the Scrum concept of a sprint backlog. This refinement ensures that each sprint is not only deliverable, but also ready for implementation within weeks, or a month at most. Such sprints transition specifications from the phases of the Architecture Development Method (specifically, phases A–E) into tangible implementations. The focus is on fostering integrated teams and environments that ensure that design, build, implementation, and operations processes interact seamlessly. This approach facilitates continuous integration and implementation, efficiently delivering each unit of the Minimum Viable Product.

The culmination of this process is observed in Phase G (Implementation Governance). This phase is critical in confirming that the agreed contracts have delivered the expected capabilities in accordance with the contractual agreements. It also ensures that all necessary information required to operate and modify the product, solution, or service is meticulously created, stored, and made accessible. This meticulous approach supports faster and higher-quality changes in the future.

Phase H (Architecture Change Management) plays a critical role in confirming that the anticipated benefits of a capability enhancement have been realized. This phase is about evaluating whether the changes implemented have tangibly improved operational and business performance. It's the moment of truth to assess whether the value propositions have been realized and whether the end users are truly satisfied with the results.

18.4.2. ADM Phases and Agile Sprints

Going through the phases of the Architecture Development Method ensures that an agile architecture can be created. It is important to develop a *Minimum Viable Architecture* during each iteration.

Each Minimum Viable Architecture created by the Enterprise Architecture team can contain some or all Architecture Development Method phases. Creating partitions (which are the equivalent of Minimum Viable Architectures) of the architecture is already part of Architecture Development Method iteration. Figure 18-13 lists the Architecture Development Method phases and describes how they can be adapted in a sprinting environment. The Preliminary Phase has been intentionally omitted.

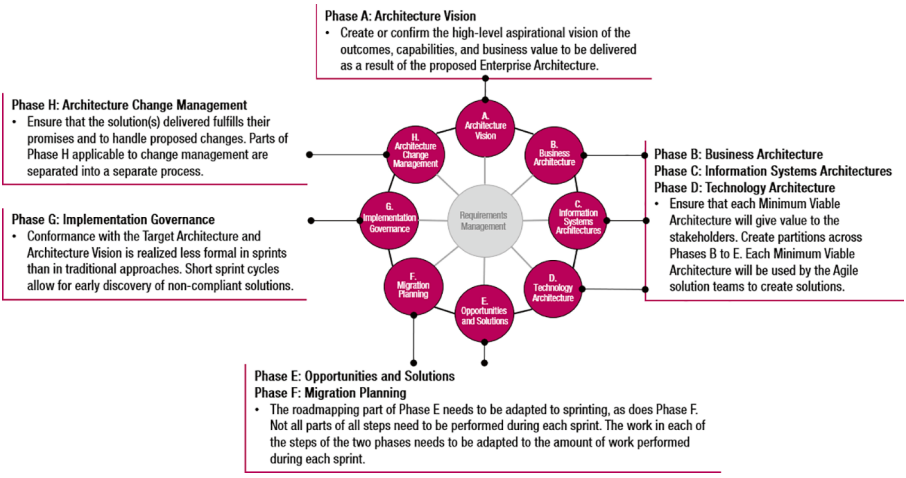


Figure 18-13. *ADM Phases and sprints*

Phase A (Architecture Vision): Start by outlining a high-level vision of what the organization aims to achieve with the proposed Enterprise Architecture. This includes identifying the desired outcomes, capabilities, and the business value expected from the initiative. Recognize that this vision is not set in stone. As the business and technology environments evolve, and as feedback is received from stakeholders, the vision may need to adapt. However, these changes are typically evolutionary, not revolutionary. This means that there may be a gradual development or change in the vision over a period of time. It does not mean that there will be a sudden, complete, or radical change in vision.

Develop the vision in a way that allows for future growth and refinement. It should serve as a stable foundation upon which the Enterprise Architecture can be built and expanded over time. Ensure that the vision offers clear direction to the teams involved. It should guide their efforts within the defined scope and timeframe, helping them align their work with the overarching goals of the Enterprise Architecture.

Phase B (Business Architecture), Phase C (Information Systems Architectures), Phase D (Technology Architecture): Instead of

attempting to design the entire architecture at once, break it down into smaller, manageable parts called Minimum Viable Architectures. Each Minimum Viable Architecture should deliver specific value to stakeholders and can be developed independently.

Divide the overall architecture work into partitions that align with the Minimum Viable Architectures. This allows multiple teams to work in parallel, each focusing on a specific Minimum Viable Architecture. Such partitioning promotes agility and faster delivery of solutions.

Ensure that Business Architects, Enterprise Architects, and solution teams collaborate closely when creating Minimum Viable Architectures and their corresponding Minimum Viable Business Designs.

Table 18-2. *Steps to adapt Phase E (Opportunities and Solutions) for Agile Sprints*

Step	Description
Understand stakeholder readiness	At the start, assess how ready stakeholders are for change. If the same stakeholders are involved throughout, this might only need to be done once.
Identify business constraints	Before each sprint, check for any new business constraints that could affect implementation. Also, review constraints at the end of a series of sprints to understand inter-team dependencies.
Review gap analysis	Regularly revisit the gap analysis to ensure it is still accurate and relevant. Refer to Chapter 14, Section 14.5.
Manage requirements across functions	In each sprint, review and manage requirements that span multiple business areas.

(*continued*)

Table 18-2. (*continued*)

Step	Description
Ensure interoperability	Confirm that the solutions developed in the sprint can work together seamlessly. Refer to Chapter 14, Section 14.6.
Refine dependencies	Identify and validate dependencies between agile teams to ensure smooth collaboration.
Assess readiness and risks	Determine if the organization is ready for the Minimum Viable Product and identify any risks. Refer to Chapter 14, Section 14.8.
Develop implementation strategy	Create a strategy for implementing the architecture that aligns with the sprint's goals.
Define work packages	List the major tasks (work packages) needed for the sprint, which will feed into team backlogs. Refer to Chapter 12, Section 12.8.3.
Plan Transition Architectures	Decide how the sprint's outputs will be transitioned to all users or a select group.
Create roadmap and migration plan	Collaborate with product owners and agile teams to define the backlog and plan for implementation. Refer to Chapter 12, Section 12.8.3.

This collaboration ensures a common understanding and alignment with business objectives.

The TOGAF Standard doesn't mandate a strict sequence for phases B–E. Feel free to iterate across these phases as needed, adapting to the project's and organization's specific requirements. This flexibility supports continuous improvement and responsiveness to change.

Phase E (Opportunities and Solutions), Phase F (Migration Planning): Not every activity needs to happen in every sprint. Therefore, the work can be tailored to match the sprint's scope and focus. When multiple solution teams are involved, coordination becomes more complex, requiring careful planning.

Phase E (Opportunities and Solutions) can be adapted for sprints by following the steps detailed in Table 18-2.

Possible adaptations for Phase F (Migration Planning) are listed in Table 18-3.

Table 18-3. *Steps to adapt Phase F (Migration Planning) for Agile Sprints*

Step	Description
Confirm management framework	Ensure that the management structures are in place to support the migration plan. This is typically done before starting the agile journey.
Assign business value	For each sprint, assign a business value to the work packages to prioritize efforts (see Chapter 12, Section 12.9.2).
Estimate resources and timelines	During sprint planning, estimate the resources needed, timelines, and delivery methods.
Prioritize projects	Use cost/benefit analysis and risk assessment to prioritize migration projects for each sprint.
Update roadmap and Architecture Definition Document	After each sprint, update the overall roadmap and the Architecture Definition Document (see Chapter 12, Section 12.2) to reflect progress.
Complete migration plan	Finalize the plan for implementing the next sprint's work, effectively updating the backlog.
Document lessons learned	At the end of each sprint, document what was learned to improve future sprints.

Phase G (Implementation Governance): When it comes to Phase G – with its objective of making sure everything aligns with the Target Architecture and the architecture vision – things work a little differently in a sprint-based approach. First, the entire compliance process becomes much less formal. And second, the short sprint cycles are a game changer. They allow non-compliant solutions to be identified early on.

Phase G (Implementation Governance) can be further adapted by following the suggested steps in Table 18-4.

Table 18-4. *Steps to adapt Phase G (Implementation Governance) for Agile Sprints*

Step	Description
Confirm scope and priorities	This is usually done at the sprint start, when the sprint goal and sprint scope are defined.
Identify deployment resources and skills	This is usually done at the beginning of each sprint for each implementation.
Guide development of solutions deployment	This is an informal process for guiding development teams through implementation.
Perform architecture compliance reviews	The difference with the traditional approach is how the compliance review is performed. This is an ongoing process, as opposed to the traditional gate/review process at a specific point in time.
Implement business and IT operations	After each sprint with an approved demo, a solution increment can be deployed.
Perform post-implementation review and close the implementation	Because reviews in sprints are an ongoing process, rather than end-user test-style reviews, this step can be the formal summary of all previous reviews of portions of the implementation.

Phase H (Architecture Change Management): The main objective of Phase H is to ensure that the solution(s) delivered fulfills its promises and to handle proposed changes. It is suggested that the parts of Phase H applicable to change management are separated into a separate process:

- **Establish a Value Realization Process:** Define a process to ensure that the business value produced during the sprint is received.
- **Deploy Monitoring Tools:** Deploy the process and associated tools to measure the business value.

Phase H (Architecture Change Management) can be further adapted by following the suggested steps in Table 18-5. These steps are applicable specifically to change management and are separated into a separate process. This process runs in parallel with agile sprints.

Table 18-5. *Steps to adapt Phase H (Architecture Change Management) for Agile Sprints*

Step	Description
Manage risks	Collect risks from demos and retrospectives and create mitigation activities.
Provide analysis for architecture change management	Analyze business value monitoring results. Evaluate other incoming change requests and move them into the business change backlog.
Develop change requests to meet performance goals	Define change requests to address poor business value creation and risk mitigation.

(*continued*)

Table 18-5. (*continued*)

Step	Description
Manage governance process	Part of the sprint work. The business team governs the Enterprise Architecture team, and the Enterprise Architecture team governs the solution team, and vice versa.
Activate the change implementation process	This is part of the normal planning work for a sprint.

Sprints are a great way to create Enterprise Architecture deliverables. It all starts with *sprint zero*, which lays the groundwork. After that, each sprint runs like clockwork, delivering a Minimum Viable Architecture each time.

At each *sprint demo*, the Enterprise Architecture team presents what they've built. This gives stakeholders a clear picture of where things stand and what's coming. And at every *retrospective*, the team steps back and reviews two things: the quality of the architecture itself and how well the process is working. It's about making sure that both the work and the way it's done are getting better and better. Measuring speed – such as tracking how requirements were allocated and how accurate effort estimates were – can really help.

When it's time to plan the next sprint, the team picks the highest-value items from the business change backlog. It's all about delivering maximum impact with each sprint. And if the workload demands it, multiple Enterprise Architecture teams can work in parallel to move even faster.

After several sprints, the architecture starts to come together, and implementation can begin. Depending on the approach, solution teams may even start building while some parts of the architecture are still being finalized. That's why decoupled design is so important. It allows teams to move forward without getting stuck waiting for each other.

When it's time to deliver solutions, agile is a great fit. Whatever method is chosen, make sure that the governance approach adapts to support it. The best path depends on where the organization is in its maturity journey.



Start with Sprint Zero: Begin with an initial preparation sprint, often called sprint zero. This sprint sets the foundation for the work ahead. Once completed, its outcomes are handled just like the results of any other sprint.

Deliver a Minimum Viable Architecture: Each sprint should focus on producing a Minimum Viable Architecture – a usable and understandable version of the architecture that can evolve over time.

Demonstrate Progress to Stakeholders: At the end of every sprint, the Enterprise Architecture team should present the results to stakeholders. This ensures that everyone is aligned and understands what is being developed.

Inspect and Adapt: After the sprint demo, hold a retrospective. Review both the quality of the Enterprise Architecture deliverables and how the team performed during the sprint. Look at key indicators like team velocity, accuracy of effort estimates, and how well requirements were allocated. Use the findings to improve future sprints.

Plan the Next Sprint: When planning each new sprint, select the most valuable and impactful items from the business change backlog. If needed, multiple Enterprise Architecture sprint teams can work in parallel to accelerate progress.

Transition to Implementation: After completing a series of sprints, the architecture is ready for implementation. In many cases, solution teams can start building before the full architecture is finished, as long as the necessary parts are defined and independent. This is where a decoupled design strategy shows its value.

Support and Govern: Throughout the implementation phase, the Enterprise Architecture team, together with the business, provides support and governance to ensure the solutions align with the architectural vision.

Adapt the Delivery Approach: Solutions can be built using agile methods or other approaches, depending on what best fits the organization's maturity and way of working. The governance approach should adapt accordingly.

Sprint planning isn't just about tasks and schedules. It's also a great time for Enterprise Architecture to sharpen desired outcomes, define measurable success, and lock down both functional and non-functional requirements. Throughout each sprint, architects should stay close to the business and solution teams to provide support and guidance. A successful iteration depends on collaboration, clarity, and course correction.



Further Reading

More information on using an agile approach in combination with the TOGAF Standard can be found in the Series Guides "Enabling Enterprise Agility", document number G20F, and "Applying the TOGAF® ADM using Agile Sprints", document number G210.

Additional information about modeling agile architecture can be found at the Portfolio of Digital Open Standards, in the document "Agile Architecture Modeling Using the ArchiMate® Language", document number G20E.

18.5. Summary

At its core, agility is about one thing: adaptability. It's a mindset built around expecting change, embracing it, and pivoting when necessary. In practice, it means working in short, focused cycles, constantly re-evaluating priorities, and ensuring that the work being done remains relevant and valuable in a world that won't sit still.

Agility puts results first. It means doing the things that create real value, not just staying busy. Rather than getting bogged down in endless plans and documentation, agility encourages organizations to focus on the results that matter most and to change course quickly when necessary. It's a culture of action, experimentation, and rapid learning. Less theory, more doing.

Building an agile architecture follows the same philosophy. And to do it well, a few key ideas come into play.

The TOGAF Standard introduces a simple but powerful model for managing architecture development at different levels of detail:

- **Strategic Architectures** set the big picture. They provide a high-level view across the whole organization, helping leadership steer the ship and shape future operations.

- **Segment Architectures** zoom in a little closer. They organize and align change efforts within specific areas, programs, or domains.
- **Capability Architectures** go deep. They describe how to deliver a particular capability, breaking it down into manageable pieces that can be grouped into programs and projects.

A smart approach is to divide the work into smaller, logical chunks. This is called architecture partitioning. Think of it as breaking the big picture into bite-sized chunks that can be tackled individually. Each partition can be developed independently and deliver real business value on its own. They can be developed in sprints. These are short, time-bound cycles designed to deliver working solutions quickly. In agile terms, the minimum amount of architecture needed to deliver value is called *Minimum Viable Architecture*. It's about doing just enough to move forward, not trying to design everything up front. Simply put, agile architecture is about making progress in manageable chunks. Traditional architecture can sometimes feel like a long, slow process. Agile turns that on its head.

In an agile way of working, tasks don't have to be done one after the other. The goal is to make parallel progress while respecting obvious dependencies. It's a balancing act. Some pieces need to be built before others, but the focus is always on moving fast and delivering value early.

The TOGAF Standard fully supports an agile way of working. The Architecture Development Method is flexible and aligns well with agile principles. In fact, each phase of the Architecture Development Method can easily be aligned with agile sprints and iterative delivery:

- **Strategic Level:** Focus on the *Preliminary Phase* and *Phase A* to set the high-level direction and outline strategic plans (also known as courses of action).

- **Segment Level:** Organize the strategic plans into actionable parts across different organizational units. Depending on the need, *phases B, C, and D* might be explored more deeply here.
- **Capability Level:** Dig into *phases B, C, and D* to create detailed Solution Architectures. These get developed further through *phases E and F* into actual Solution Building Blocks.

All of this leads naturally to *Phase G (Implementation Governance)*, where sprint-based development allows for early identification of non-compliant solutions and rapid course correction when needed.

Finally, *Phase H (Architecture Change Management)* ensures that the delivered solutions do their job: deliver real, measurable value. In an agile environment, change management activities can run in parallel with sprints, keeping momentum high and ensuring that the architecture continues to evolve in line with real-world needs.